



Tahar Issad - Portfolio

Full-Stack Developer/Software Engineer



I am a web developer with a passion for creating modern, responsive, and user-friendly interfaces. Currently expanding my skills in cybersecurity and preparing for the ISC2 certification. I enjoy turning ideas into projects and constantly learning new technologies.

My Projects

Here are some of the projects I have worked on, showcasing my skills in C++, Node.js, React and more. Each project demonstrates my ability to solve complex problems and deliver high-quality software solutions.



Remote Pong Game (Transcendence at 42)

Transcendence is a collaborative project from the 42 curriculum, aimed at creating a fully functional online gaming platform clone. It features user management, secure authentication, real-time player communication, and a live match system with scores and statistics. The project combines both front-end and back-end development to deliver a complete interactive experience.

[React](#)[Node.js](#)[TypeScript](#)[Fastify](#)[WebSockets](#)[PostgreSQL](#)[Redis](#)[Git / GitHub](#)[Docker/Doker Compose](#)[Hashicorp Vault](#) [GitHub](#)

LoiHub

LoiHub is an AI-powered web application that reimagines the legislative process as a GitHub-like experience. It allows users to browse laws, track amendments as commits, compare legislative revisions, explore timelines, and generate AI summaries from official French National Assembly datasets. The project combines modern full-stack technologies with data engineering and AI to make legislation more transparent and accessible.

[React](#)[next.js](#)[TypeScript](#)[PostgreSQL](#)[Docker](#)[Git / GitHub](#) [GitHub](#)



Personal Portfolio Website

A responsive personal portfolio built with Next.js, React, and TailwindCSS, showcasing projects, skills, and contact functionality. Fully front-end, optimized for GitHub Pages deployment.

[Next.js](#)[React](#)[TypeScript](#)[TailwindCSS](#)[GitHub Pages](#) [GitHub](#)

ft_linear_regression

An introduction to artificial intelligence and machine learning is provided through the implementation of a basic linear regression model. A program is developed to predict car prices using a linear function trained with a gradient descent algorithm. The project covers data handling, cost functions, parameter optimization, and model training, laying the foundation for further studies in machine learning and applied mathematics.

[Python](#)[Machine Learning](#)[Linear Regression](#)[Gradient Descent](#) [GitHub](#)



Webserv

A custom HTTP server is implemented from scratch, following the HTTP/1.1 specification. The server is designed to handle client requests, manage connections, and serve web content that can be tested using a real web browser. Core concepts such as request parsing, response generation, status codes, and server configuration are explored. This project provides a deep understanding of how web servers operate internally and of the HTTP protocol, one of the most widely used protocols on the internet.

[C++](#)[HTTP/1.1](#)[Networking](#)[Server Implementation](#)[Sockets](#)[GitHub](#)

Inception

A complete containerized infrastructure is designed and deployed using Docker and Docker Compose. Multiple services are configured and orchestrated, including NGINX with SSL/TLS, WordPress with PHP-FPM, and MariaDB. The project focuses on container networking, volume management, environment isolation, and secure service configuration. All services are deployed within a personal virtual machine, reinforcing best practices in system administration, containerization, and secure web service deployment.

[Docker](#)[Docker Compose](#)[NGINX](#)[WordPress](#)[MariaDB](#)[GitHub](#)



cub3d

A ray-casting engine inspired by early 3D first-person games is developed. The project focuses on rendering a real-time, first-person perspective inside a maze by Player movement and rotation are implemented with collision detection to ensure realistic navigation within the environment. The engine handles keyboard input, map parsing, texture loading, and screen rendering using MiniLibX. Performance considerations such as efficient ray calculation and frame updates are addressed to maintain smooth gameplay. This project reinforces concepts in computer graphics, trigonometry, and low-level optimization while strengthening understanding of game engine fundamentals.

[C](#)[MiniLibX](#)[Computer graphics](#)[Trigonometry](#)[Game development](#)[!\[\]\(0b5e7e25e8775f7e7e80906ada4f0021_img.jpg\) GitHub](#)

NetPractice

Core computer networking fundamentals are taught through hands-on problem-solving. TCP/IP addressing, subnetting, routing, default gateways, and OSI layers are practiced using interactive network diagrams.

[Networking](#)[TCP/IP](#)[Subnetting](#)[Routing](#)[OSI Layers](#)[!\[\]\(e50091943b385fe16d3277389202856f_img.jpg\) GitHub](#)



Minishell

A minimal UNIX shell is developed, closely replicating the behavior of a standard shell. The project involves parsing user input, handling command execution, and managing processes using system calls such as `fork`, `execve`, and `wait`. Input is tokenized and interpreted to support command sequences, pipes, and redirections. Signal handling is implemented to manage user interactions (`Ctrl-C`, `Ctrl-D`, `Ctrl-I`) in an interactive environment. Several built-in commands, such as `cd`, `echo`, `pwd`, `export`, `unset`, and `exit`, are implemented to behave consistently with their UNIX counterparts. Environment variables are managed dynamically, and error handling is carefully designed to ensure robust and predictable shell behavior. The project emphasizes low-level system programming, process control, and a deep understanding of UNIX principles.

[C](#)[UNIX](#)[System programming](#)[Process control](#)[Signal handling](#)[Git / GitHub](#) [GitHub](#)

Philosophers

Concurrent programming concepts are introduced, with a focus on multithreading and multiprocessing. Synchronization issues such as race conditions and deadlocks are addressed.

[C](#)[Multithreading](#)[Multiprocessing](#)[Synchronization](#)[Rigor](#) [GitHub](#)



push_swap

Data is sorted within a stack using a limited set of instructions while minimizing the number of operations. Various sorting algorithms are explored and optimized based on the input size.

[C](#)[Algorithms](#)[Rigor](#)[Algorithms & AI](#) [GitHub](#)

fract-ol

Two-dimensional graphical programming is explored through the visualization of fractals. The project uses MiniLibX to render mathematical fractals and handle graphical events.

[C](#)[MiniLibX](#)[Mathematics](#)[Graphics](#) [GitHub](#)

pipex

A deeper understanding of UNIX concepts such as redirections and pipes is developed. The project serves as an introduction to more advanced UNIX-based projects later in the curriculum.

[Algorithms](#)[Rigor](#)[Imperative programming](#) [GitHub](#)

Born2beroot

An introduction to system administration and virtualization is provided. The project focuses on installing and configuring a secure virtual machine while following strict security and system configuration rules.

[Rigor](#)[System administration](#)[Virtualization](#)[Unix](#)[network](#) [GitHub](#)



ft_printf

The standard C printf function is recoded. Through this project, variadic functions are studied and implemented. Once validated, the function is reused in subsequent projects.

Rigor

Imperative programming

Algorithms

Unix

 [GitHub](#)



Get Next Line

A function is implemented to read content line by line from a file descriptor, whether the input comes from a file, standard input, or a future network connection. This function is designed to be reused throughout later projects.

Rigor

Imperative programming

Algorithms

Git / GitHub

Unix

 [GitHub](#)



LBFT

This project is the very first project undertaken as a learner at 42. A few functions from the C standard library must be recoded, along with additional utility functions that will be used throughout the entire curriculum.

Rigor

Imperative programming

Algorithms

Unix

 [GitHub](#)

Contact Me

Feel free to reach out to me for any colab, inquiries or opportunities.

Information

-  **Email**
issad.tahar@icloud.com
-  **Location**
Paris, ile-de-France, France
-  **Open to work**
stage, full-time

Find me on



Name

Your name

Email

eg: john.doe@example.com

Message

Your message

Tahar Issad

Full stack developer, software engineer, and lifelong learner. Building scalable and user-friendly web applications. Using modern technologies to solve real-world problems and create impactful solutions.

Navigation

About
Projects
Contact
CV

Social Media



© 2026 Tahar Issad. All rights reserved.

Designed and developed by Tahar Issad using Next.js and Tailwind CSS.